

Loan Approval Prediction API

A machine learning-powered REST API that predicts whether a loan application is likely to be approved based on applicant and loan-related information.

The project combines a trained machine learning pipeline with **FastAPI, JWT authentication, PostgreSQL, SQLAlchemy, Alembic, Docker, and cloud deployment**.

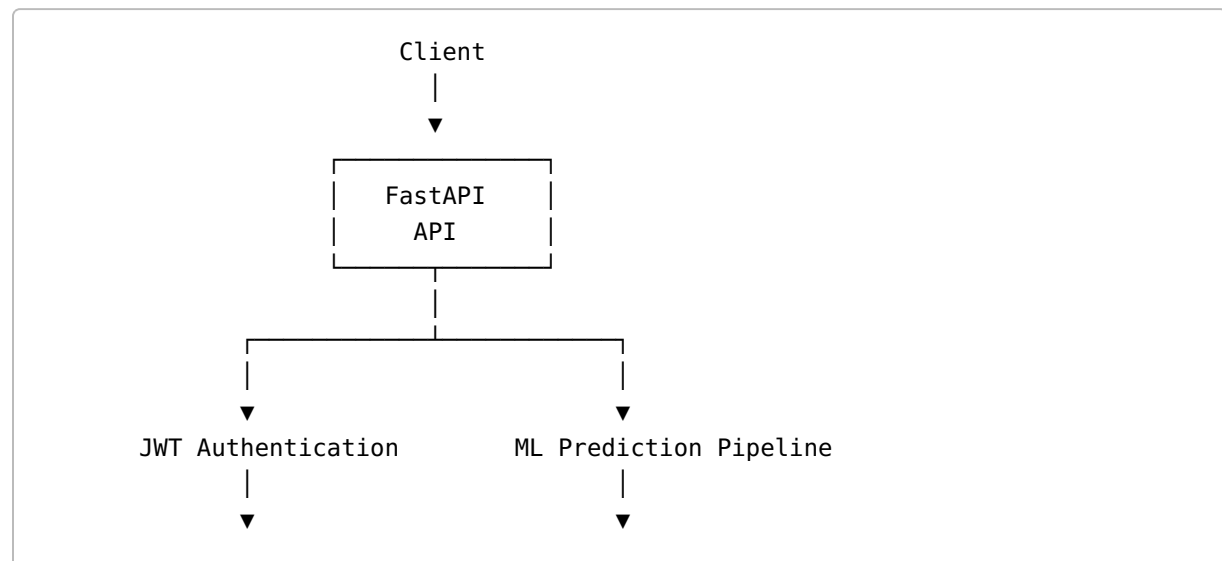
Project Overview

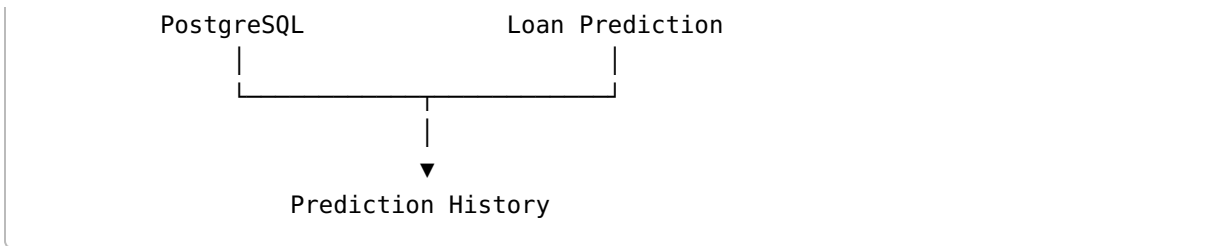
The application accepts loan application details and uses a trained classification model to predict the loan approval status.

It also provides:

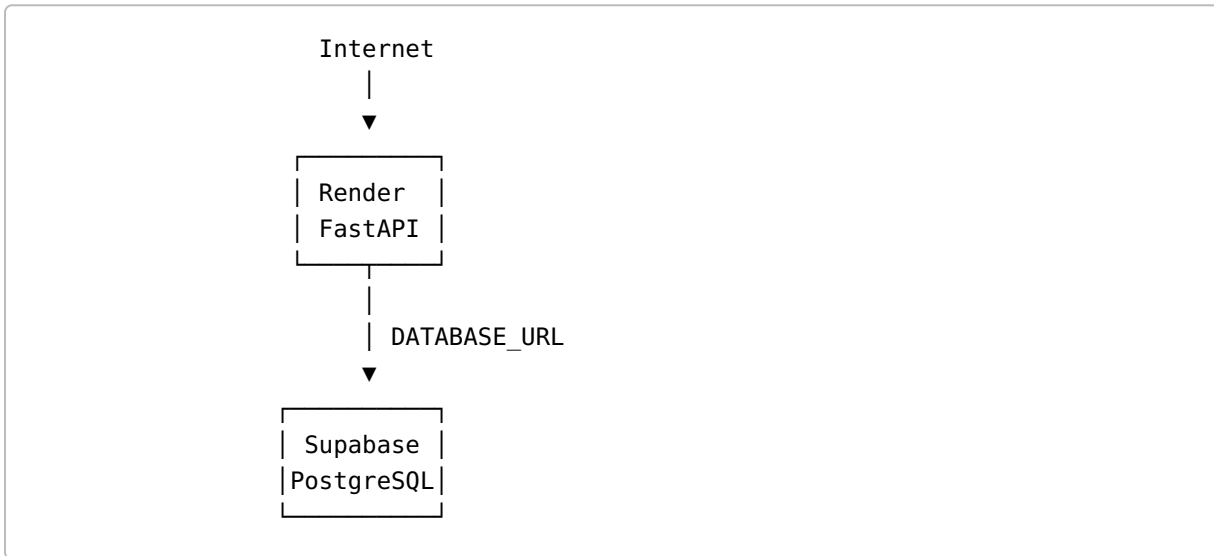
- User registration and authentication
 - JWT-based authorization
 - Loan approval prediction
 - Prediction probability
 - Prediction history stored in PostgreSQL
 - CSV-based prediction support
 - Automatic database migrations using Alembic
 - Dockerized application and PostgreSQL database
 - Interactive API documentation using Swagger UI
-

Architecture





Deployment Architecture



Machine Learning

Dataset

The model is trained on loan application data containing features such as:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Coapplicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

`Loan_ID` is used to identify the application but is not used as a predictive feature.

Data Preprocessing

The ML pipeline handles preprocessing before model prediction.

Numerical Features

- Missing values handled using appropriate imputation
- Standard scaling applied

Categorical Features

- Missing values replaced with `"unknown"`
- One-hot encoding applied
- Unknown categories handled using `handle_unknown="ignore"`

Loan Amount Term

Missing values are handled using the most frequent value before scaling.

This preprocessing is included in the saved ML pipeline so that the same transformations are automatically applied during API prediction.

Model

Multiple classification algorithms were evaluated during model development, including:

- Logistic Regression
- Random Forest
- Decision Tree
- Support Vector Classifier
- K-Nearest Neighbors
- Gradient Boosting

The selected model is saved as:

```
loan_prediction_model.joblib
```

The complete preprocessing and prediction pipeline is loaded by the FastAPI application.

Authentication

The API uses **JWT-based authentication**.

Authentication flow

```
Register
  ↓
User stored in PostgreSQL
  ↓
Login
  ↓
Username + Password verification
  ↓
JWT access token
  ↓
Bearer token
  ↓
Protected prediction endpoints
```

Passwords are securely hashed before being stored in the database.

Protected endpoints retrieve the authenticated user from the JWT rather than trusting user information sent by the client.

Database

PostgreSQL is used to store users and prediction history.

Database Tables

`users`

Stores registered users.

```
user_id
username
password
```

`prediction_history`

Stores prediction information associated with the authenticated user.

```
Loan_ID
user_id
ApplicantIncome
```

```
LoanAmount
Credit_History
loan_status
approval_probability
created_at
```

A foreign key connects:

```
prediction_history.user_id
      ↓
users.user_id
```

Database Migrations

Alembic is used for database schema migrations.

The application does not rely on manually creating database tables.

On a fresh deployment, the migration command creates the required schema:

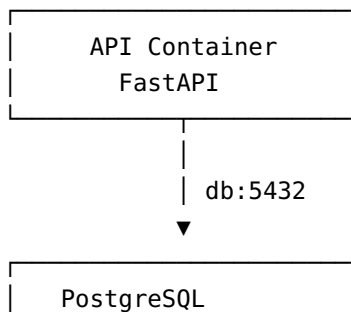
```
alembic upgrade head
```

The Docker container automatically runs migrations before starting the FastAPI server.

Docker

The project is containerized using Docker.

Docker Compose runs:



Container

Start the project locally

```
docker compose up --build
```

The API will be available at:

```
http://localhost:8000
```

Swagger documentation:

```
http://localhost:8000/docs
```

API Endpoints

Authentication

Method	Endpoint	Description
POST	<code>/register</code>	Register a new user
POST	<code>/login</code>	Authenticate user and generate JWT

Prediction

Method	Endpoint	Description
POST	<code>/predict</code>	Predict loan approval
POST	<code>/predict-file</code>	Predict loan approvals from CSV

Protected prediction endpoints require:

```
Authorization: Bearer <JWT_TOKEN>
```

Example Prediction Request

```
{
  "Loan_ID": 1001,
  "Gender": "Male",
  "Married": "Yes",
  "Dependents": "0",
  "Education": "Graduate",
  "Self_Employed": "No",
  "ApplicantIncome": 5000,
  "CoapplicantIncome": 1500,
  "LoanAmount": 150,
  "Loan_Amount_Term": 360,
  "Credit_History": 1,
  "Property_Area": "Urban"
}
```

Example Response

```
{
  "loan_status": "Y",
  "approval_probability": "70%"
}
```

The prediction is also stored in the `prediction_history` table for the authenticated user.

Testing

Testing is planned as a future improvement and will be incorporated into subsequent projects.

Tech Stack

Machine Learning

- Python
- Pandas
- NumPy
- Scikit-learn
- Joblib

Backend

- FastAPI
- Uvicorn
- Pydantic

Authentication & Security

- JWT
- PyJWT
- Argon2 password hashing

Database

- PostgreSQL
- SQLAlchemy
- Alembic
- psycopg2

DevOps / Deployment

- Docker
- Docker Compose
- Git
- GitHub
- Render
- Supabase PostgreSQL

Project Structure

```
loan_prediction/
|
├─ main.py
├─ database.py
├─ models.py
├─ schemas.py
├─ auth.py
├─ requirements.txt
├─ Dockerfile
├─ docker-compose.yml
├─ .dockerignore
├─ .env
├─ alembic.ini
|
├─ alembic/
|   └─ env.py
```

```
|   └─ versions/
|
| └─ router/
|   └─ auth_router.py
|   └─ loan_prediction_router.py
|
| └─ services/
|   └─ loan_prediction_service.py
|
| └─ tests/
|
| └─ loan_prediction_model.joblib
```

Environment Variables

The application uses environment variables for configuration.

Example:

```
DATABASE_URL=postgresql://username:password@host:port/database
SECRET_KEY=your_secret_key
```

Sensitive configuration such as passwords and secret keys should not be committed to GitHub.

The `.env` file is excluded through `.gitignore` and `.dockerignore`.

Running Locally

1. Clone the repository

```
git clone <your-github-repository-url>
cd loan_prediction
```

2. Create and activate a virtual environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure environment variables

Create a `.env` file:

```
DATABASE_URL=your_database_url  
SECRET_KEY=your_secret_key
```

5. Run database migrations

```
alembic upgrade head
```

6. Start FastAPI

```
uvicorn main:app --reload
```

Open:

```
http://localhost:8000/docs
```

Deployment

The application can be deployed using:

- **Render** — FastAPI application
- **Supabase** — PostgreSQL database

The production database URL is supplied through environment variables, allowing the same application code to work across local and cloud environments.

Project Goals

This project was built to demonstrate practical experience in:

- Machine learning model development
 - Data preprocessing and model pipelines
 - REST API development
 - Authentication and authorization
 - Relational database integration
 - Database migrations
 - Docker containerization
 - Cloud deployment
 - End-to-end ML application development
-

Future Improvements

Possible future improvements include:

- CI/CD using GitHub Actions
 - ML experiment tracking with MLflow
 - Improved model monitoring
 - API rate limiting
 - Automated model retraining
 - More comprehensive test coverage
 - Kubernetes-based deployment
-

Author

Anirudha Hazra

This project demonstrates an end-to-end machine learning application developed with Python, FastAPI, PostgreSQL, Docker, and cloud deployment.